Modelo do caderno

O que escrever no caderno para a Questão 1 (HeapSort), item por item. Copie a **caneta** no meio da folha; o **lápis na margem** são lembretes curtos; passe **marca-texto** nos resultados.

caneta (o conteúdo) · lápis na margem (lembretes) · marca-texto (resultados)

FOLHA 1 · ITEM A

a) Funcionamento do HeapSort

Heap de máximo (vetor A[1..n] visto como árvore)

índices — pai(i) = ⌊i/2⌋ · esq(i) = 2i · dir(i) = 2i + 1

propriedade — A[pai(i)] ≥ A[i] para todo i → o maior fica na raiz A[1]

maxHeapify(A, i, tam)

"desce" — compara A[i] com os filhos; se um filho é maior, troca com o maior

e continua descendo a partir dele. Supõe que os filhos já são heaps.

construirMaxHeap(A, n)

folhas já são heaps — i de ⌊n/2⌋ até 1: chama maxHeapify(A, i, n)

de baixo para cima → no fim, A inteiro é heap de máximo

heapSort(A, n)

1. constrói o heap (maior na raiz)

maior vai pro fim — 2. para i = n até 2: troca A[1] com A[i] (maior vai pro fim),

tam = tam − 1 e maxHeapify(A, 1, tam) conserta a raiz

3. A[i..n] vai ficando ordenado da direita para a esquerda

b) Custo do maxHeapify

cada chamada:

2 comparações + no máx. 1 troca → custo constante c

e no máximo 1 chamada recursiva, um nível abaixo

$$T(h) = T(h-1) + c, \quad T(0) = c$$

abrindo

$T(h) = T(h-2) + 2c = \ldots = T(0) + h \cdot c = c(h+1)$

$$\text{maxHeapify} = O(h)$$

Altura de um heap com n elementos

árvore completa

níveis 0 até h−1 cheios: $1 + 2 + \ldots + 2^{h-1} = 2^h - 1$ nós

o nível h tem de 1 até 2^h nós:

$$2^h - 1 + 1 \leq n \leq 2^{h+1} - 1 \rightarrow 2^h \leq n < 2^{h+1}$$

$\log_2$ dos dois lados

$$h \leq \log_2 n < h + 1 \rightarrow h = \lfloor \log_2 n \rfloor$$

$$\text{maxHeapify}(A, 1, n) = O(\log n)$$

c) Custo do construirMaxHeap

Análise simples

$\lfloor n/2 \rfloor$ chamadas × $O(\log n)$ cada → $O(n \log n)$

não é justo

mas a maioria dos nós está embaixo e desce pouco

Análise exata

nó de altura h

desce no máx. h níveis → custo $\leq c \cdot h$

$$\text{nº de nós de altura } h \leq \left\lceil \frac{n}{2^{h+1}} \right\rceil \leq \frac{n}{2^h}$$

$\lceil x \rceil \leq 2x$ se $x \geq ½$

(metade são folhas h=0, ¼ tem h=1, …)

$$\text{custo} \leq \sum_{h=0}^{\lfloor \log n \rfloor} \frac{n}{2^h} \cdot c \cdot h \leq c \cdot n \sum_{h=0}^{\infty} \frac{h}{2^h}$$

Quanto vale $S = \Sigma\, h/2^h$?

$S = 1/2 + 2/4 + 3/8 + 4/16 + \ldots$

divide por 2

$S/2 = \quad 1/4 + 2/8 + 3/16 + \ldots$

subtrai

$S - S/2 = 1/2 + 1/4 + 1/8 + \ldots = 1$ → **$S = 2$**

custo $\leq c \cdot n \cdot 2 = 2cn$ → $O(n)$

$\geq n/2$ chamadas

custo $\geq (n/2) \cdot c$ → $\Omega(n)$

construirMaxHeap = $\Theta(n)$

d) Pior caso do HeapSort

$T(n) = \text{construirMaxHeap} + (n-1) \times [\ \text{troca} + \text{maxHeapify}(A, 1, \text{tam})\ ]$

$$T(n) \leq c_1 n + \sum_{\text{tam}=1}^{n-1} (\ c_2 + c_3 \log_2 \text{tam}\ )$$

$\log_2 \text{tam} \leq \log_2 n$

$T(n) \leq c_1 n + c_2 n + c_3 (n-1) \log_2 n \ \rightarrow\ O(n \log n)$

Limite inferior

árvore de decisão

todo algoritmo por comparação precisa, no pior caso, de pelo menos $\log_2(n!)$ comparações ($n!$ ordens possíveis)

metade dos fatores são $\geq n/2$

$n! \geq (n/2)^{n/2} \ \rightarrow\ \log_2(n!) \geq (n/2) \cdot \log_2(n/2) \ \rightarrow\ \Omega(n \log n)$

Pior caso: $T(n) = \Theta(n \log n)$

e) Melhor caso

Vetor com todos os elementos iguais

"<" estrito — A[esq] < A[maior] e A[dir] < A[maior] nunca são verdadeiros

→ maior = i → maxHeapify para na hora: custo c (constante)

construirMaxHeap: $\lfloor n/2 \rfloor \cdot c = \Theta(n)$

ordenação: $(n - 1) \cdot (\text{troca} + c) = \Theta(n)$

Melhor caso (todos iguais): $\Theta(n)$

Vetor já ordenado ajuda?

crescente — construirMaxHeap faz trocas (o menor está na raiz)

decrescente — já é heap → construir sem trocas, mas ...

folha pequena — em cada iteração a raiz recebe A[i], que veio do último

vai pra raiz — nível e é pequeno → desce até ≈ log n níveis

→ com elementos distintos o melhor caso continua $\Theta(n \log n)$

f) Caso médio

melhor caso (distintos) $\leq$ caso médio $\leq$ pior caso

$\Theta(n \log n)$ $\Theta(n \log n)$

média $\leq$ pior

$O(n \log n)$ por cima

árvore de decisão

por comparação, a média também precisa de $\geq \log_2(n!) \rightarrow \Omega(n \log n)$

Caso médio: $\Theta(n \log n)$

Comparação

caso	complexidade
melhor (todos iguais)	$\Theta(n)$
melhor (distintos)	$\Theta(n \log n)$
médio	$\Theta(n \log n)$
pior	$\Theta(n \log n)$

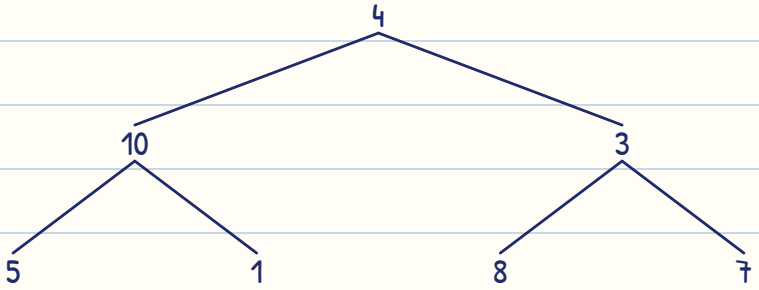
$\neq$ QuickSort

não existe caso $\Theta(n^2)$: o heap sempre tem altura log n

g) Exemplo: A = [4, 10, 3, 5, 1, 8, 7]

Árvore inicial (n = 7)

nível 0 / nível 1 / nível 2

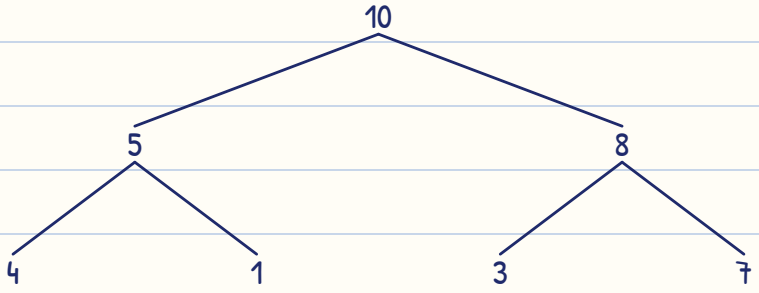


construirMaxHeap: i = ⌊7/2⌋ = 3, 2, 1

- *i = 3* — 3 < 8 → troca(3, 6) → [4, 10, 8, 5, 1, 3, 7]
- *i = 2* — 10 ≥ 5 e 1 → nada
- *i = 1* — 4 < 10 → troca(1, 2) → [10, 4, 8, 5, 1, 3, 7]
- *desce* — 4 < 5 → troca(2, 4) → [10, 5, 8, 4, 1, 3, 7]

Heap de máximo

nível 0 / nível 1 / nível 2



confere — [10, 5, 8, 4, 1, 3, 7] todo pai ≥ filhos

Fase de ordenação (| separa o heap da parte ordenada)

i	troca(1, i)	maxHeapify(1, tam)	vetor
7	[7, 5, 8, 4, 1, 3 | 10]	7<8: troca(1,3)	[8, 5, 7, 4, 1, 3 | 10]
6	[3, 5, 7, 4, 1 | 8, 10]	3<7: troca(1,3)	[7, 5, 3, 4, 1 | 8, 10]
5	[1, 5, 3, 4 | 7, 8, 10]	troca(1,2), troca(2,4)	[5, 4, 3, 1 | 7, 8, 10]
4	[1, 4, 3 | 5, 7, 8, 10]	1<4: troca(1,2)	[4, 1, 3 | 5, 7, 8, 10]
3	[3, 1 | 4, 5, 7, 8, 10]	3 ≥ 1: nada	[3, 1 | 4, 5, 7, 8, 10]
2	[1 | 3, 4, 5, 7, 8, 10]	tam = 1: nada	[1 | 3, 4, 5, 7, 8, 10]

A = [1, 3, 4, 5, 7, 8, 10]